

Supplementary Material:

SVA Suite Rationale and Categorization

Achyuta Ganesh

This supplementary document details the final 30 SystemVerilog Assertions (SVAs) chosen for the Gold-Standard verification suite. Per the evaluation rubric, each property explicitly details what it protects, what bug would slip past without it, and how a violation manifests.

A) Local State: Reset

- **P20a (Reset reads zero):**
 - Ensures the pipeline is cleanly initialized at the clock edge.
 - Without it, a dirty reset condition could leave residual values in the pipeline buffers.
 - As a result upon boot, CPU would execute garbage instructions.
- **P20b (Asynchronous clear):**
 - Ensures the CPU state clears immediately upon reset, independent of the clock.
 - Without it, a synchronous-only reset could allow a corrupted state to persist (undetected) till the clock edge atleast.

B) Load-Use Hazard → Stall

- **P1 (Happens when it must):**
 - Protects data dependencies between memory loads and immediate ALU uses.
 - Without it, the hazard detection unit could fail to recognize a dependency and let the pipeline continue.
 - This would manifest as the dependent instruction computing using stale register data instead of the newly loaded memory data.
- **P1b (Freezes PC & IF/ID):**
 - Prevents instruction fetch from advancing during a stall.
 - Without it, a broken stall signal could let the PC increment while the rest of the pipeline waits.
 - The CPU would skip instructions entirely.
- **P2 (ID/EX clears to bubble):**
 - Injects a NOP to separate the stalled instruction from the executing load.
 - Without it, a stall might freeze the front half of the pipeline but fail to clear the ID/EX control signals.
 - The instruction currently sitting in ID/EX would execute twice, corrupting architectural state.
- **P5 (Stalls exactly 1 cycle):**
 - Ensures pipeline efficiency by strictly bounding the load-use penalty.
 - Without it, a sticky hazard flag could cause a multi-cycle stall.
 - The CPU would freeze unnecessarily for multiple cycles on a single load-use hazard.

C) Pipeline Handoffs

- **P18 (PC updates correctly):**
 - Protects the fundamental sequential execution of the program.
 - Without it, the PC could skip or repeat addresses during normal operation.
- **P3 (IF/ID frozen during stall):**
 - Makes sure the contents of the IF/ID remain frozen during stall
 - The currently stalled instruction would be overwritten and permanently lost.
 - P1b checks the control wires to make sure IF/ID is frozen but this checks the actual reg data against the previous clock cycle to guarantee it froze perfectly.
- **P28 (IF/ID captures if no stall):**
 - Protects the normal propagation of fetched instructions.
 - Without it, a stuck stall flag could prevent new instructions from entering the pipeline.
 - While P3 checks that during a stall the regs are frozen this makes sure it captures the new data correctly if there is no stall.
- **P25 (Data enters ID/EX if no hazard):**
 - Protects normal instruction flow from Decode to Execute.
 - Similar to previous, P2 made sure a bubble (NOP) is inserted in the case of stall and this guarantees its working if there is no stall

D) Backpressure

- **P42 (Req maintains value):**
 - Makes sure req maintains its value until ready
 - Without it, the CPU could change the address or data midway through a wait state.
- **P43 (Mem stall freezes pipeline):**
 - Makes sure that a Mem stall correctly freezes the pipeline
 - Without it, IF, ID, and EX could advance while MEM is blocked.
 - Instructions currently in EX would overwrite the instruction stuck in MEM, destroying it.
- **P44 (Pipeline not stalled forever):**
 - Protects the liveness of the processor against deadlocks.
 - The ready signal would never assert, holding the wait state indefinitely.

E) Forwarding

- **P6 (Youngest older producer wins):**
 - Protects data dependency priorities when multiple stages hold valid data for the same register.
 - Without it, a double-data hazard could incorrectly prioritize MEM/WB over the newer EX/MEM data.
 - The ALU would compute using older, stale data instead of the most recent result.
- **P38 (Live operand data correctness):**
 - Ensures that forwarding happens when it must and the data entering the ALU during forwarding = what the reg file would hold if we did not have the pipeline
 - Without it, a wiring bug could slip past where the multiplexer makes the correct decision, but pulls data from the wrong wire

F) Branch, Commits, and Memory

- **P8 (Flushing happens when it must):**
 - Protects the control flow logic from executing mispredicted instructions.
 - The two wrong-path instructions in IF and ID would incorrectly execute and commit.
- **P8b (Flushed instr has no side effects):**
 - Protects the architectural state (reg/mem writes) from changing due to a flushed instr.
 - Without it, a flush could zero the instruction but leave RegWrite or MemWrite high.
 - A squashed instruction would still alter a register or write garbage to memory.
- **P35 (Committed value comes from right source):**
 - Makes sure the final commit value always comes from the right source (ALU/mem)
 - Without it, a lw instruction could accidentally write ALU results, or an add could write memory data.
- **P39a (Right \$rs data to memory):**
 - Makes sure the right forwarded data is written to memory
 - P38 protects forwarding to the ALU not memory.
 - A sw instruction would write stale or incorrect data into the memory.
- **P39b (Store lands at right address):**
 - Protects memory addressing calculation.
 - Without it, an ALU bug could calculate the wrong offset for the memory write.
 - The CPU would overwrite unrelated data structures in RAM.
- **P39c (No writes to RAM during stall):**
 - Protects data memory from writes while the pipeline waits.
 - Without it, MemWrite signal could stay turned on while the target memory address could randomly change during the freeze.
 - The processor would write invalid data across random memory addresses while frozen.

H) Environment & Architectural Invariants

- **P51 (\$zero is immutable):**
 - Protects the MIPS architectural guarantee that register 0 cannot be altered.
 - Without it, a software instruction like add \$0, \$1, \$2 could successfully change the hardware register.
 - Subsequent instructions reading \$zero would receive non-zero values.
- **P40 (Write-through bypass):**
 - Protects against read-after-write hazards in the same clock cycle within the register file.
 - It is basically another "forwarding"
 - The ID stage would read stale data, requiring an unnecessary pipeline stall.
- **P26 (Reading \$zero returns 0):**
 - Protects the output wires of the register file.
 - Without it, the register file could store 0 but physically output noise on the \$zero read port.
 - The ALU would receive garbage bits when attempting to use \$zero.
- **P12 (PC is multiple of 4):**
 - Protects instruction alignment requirements.
 - Without it, a branch could calculate an unaligned address (e.g., PC = 0x03).
 - The instruction memory would return misaligned, corrupted instruction bits.
- **A9 (PC bounded to IMEM):**

- Protects the formal solver from testing impossible physical limits.
 - Without it, the solver could inject out-of-bounds addresses (this is an assumption, not an assertion).
 - The solver would falsely report a hardware bug when the PC exceeds physical memory bounds.
- **P17 (Data accesses bounded to DMEM):**
 - Protects the memory array from out-of-bounds requests.
 - Without it, an ALU offset could cause a read/write beyond the top of memory.
 - The simulation would crash or return X (undefined) states.